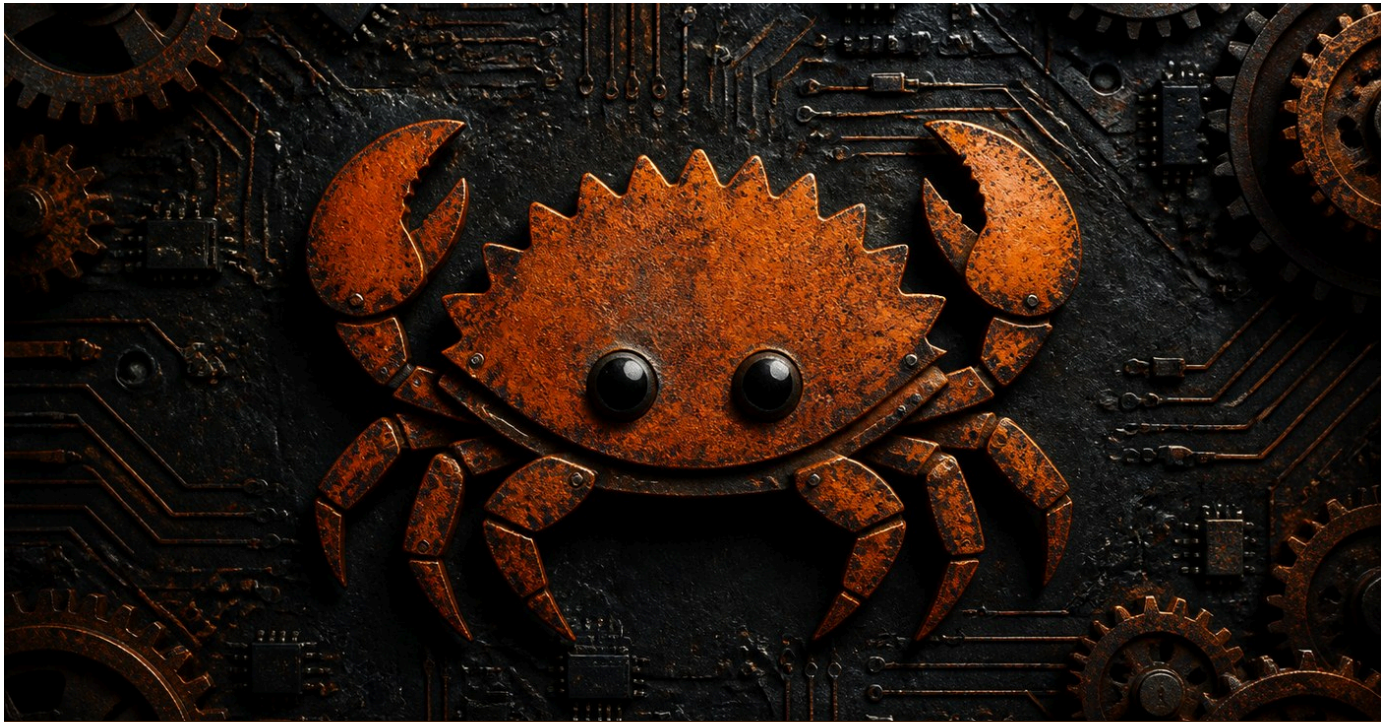




FORMATION RUST - LES BASES

Rust

Apprendre le langage systeme le plus
aime, pas a pas.

DanielCraft - Livre debutant - Pack Systeme

Sommaire

Clique un chapitre ou un sous-chapitre pour y aller.

1. C'est quoi Rust ?

- Le langage en une phrase
- Pourquoi apprendre Rust ?
- A quoi ca ressemble ?
- Petite histoire
- En vrai
- A retenir

2. Installer Rust

- Ce qu'il te faut
- Installer rustup
- Cargo : le gestionnaire de projet
- Installer VS Code
- Petite histoire
- A retenir

3. Premier programme

- Hello World
- Compiler et executer
- Affichage formate
- Les commentaires
- Petite histoire
- A retenir

4. Les variables

- Immutabilite par default
- Shadowing
- Constantes
- Conventions
- Petite histoire
- A retenir

5. Les types de donnees

- Les types scalaires
- Les strings
- Inference de type
- Conversion
- Tuples et tableaux
- Petite histoire
- A retenir

6. Les conditions

- if / else
- if comme expression
- else if

- [match](#)
- [Petite histoire](#)
- [A retenir](#)

7. [Les boucles](#)

- [loop : boucle infinie](#)
- [while](#)
- [for et ranges](#)
- [Parcourir une collection](#)
- [break et continue](#)
- [Petite histoire](#)
- [A retenir](#)

8. [Les fonctions](#)

- [Declarer une fonction](#)
- [Retourner une valeur](#)
- [Return explicite](#)
- [Closures](#)
- [Petite histoire](#)
- [A retenir](#)

9. [L'ownership](#)

- [Le concept central de Rust](#)
- [Le déplacement \(move\)](#)
- [Le clonage](#)
- [Les references \(emprunt\)](#)
- [References mutables](#)
- [Petite histoire](#)
- [A retenir](#)

10. [Les structs](#)

- [Definir une struct](#)
- [Methodes avec impl](#)
- [Tuple structs](#)
- [Derive](#)
- [Petite histoire](#)
- [A retenir](#)

11. [Enums et pattern matching](#)

- [Les enums](#)
- [Enums avec donnees](#)
- [match avec enums](#)
- [Option<T>](#)
- [if let](#)
- [Petite histoire](#)
- [A retenir](#)

12. [Gerer les erreurs](#)

- [Result<T, E>](#)

- L'opérateur ?
- unwrap et expect
- Créer ses propres erreurs
- Petite histoire
- A retenir

13. Mini-projet : outil CLI de notes

- L'objectif
- Structure (src/main.rs)
- Ce que tu apprends
- A retenir

14. Ce qu'il faut retenir

- Les briques essentielles
- La méthode
- Ce qui vient après
- A retenir

15. Atelier : variables et types

- Exercice 1 : carte d'identité
- Exercice 2 : shadowing
- Exercice 3 : conversion
- Defi : temperature
- A retenir

16. Atelier : fonctions

- Exercice 1 : salutation
- Exercice 2 : moyenne
- Exercice 3 : mot de passe valide
- Exercice 4 : division sécurisée
- A retenir

17. Atelier : structs et enums

- Exercice 1 : struct Produit
- Exercice 2 : panier
- Exercice 3 : enum Forme
- A retenir

18. Erreurs classiques en Rust

- 1. Utiliser une valeur après un move
- 2. Référence mutable + immutable
- 3. Oublier le point-virgule
- 4. unwrap() en production
- 5. Confusion &str vs String
- 6. Oublier mut
- A retenir

19. Bonnes pratiques

- Conventions Rust

- [Structure de projet](#)
- [Gestion des erreurs](#)
- [Documentation](#)
- [Outils](#)
- [A retenir](#)

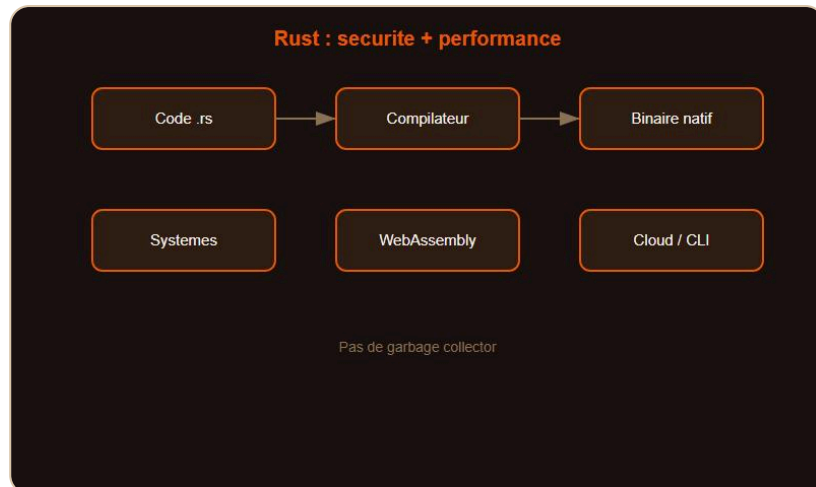
20. [Quiz Rust - Les bases](#)

- [Teste tes connaissances](#)
- [Reponses](#)

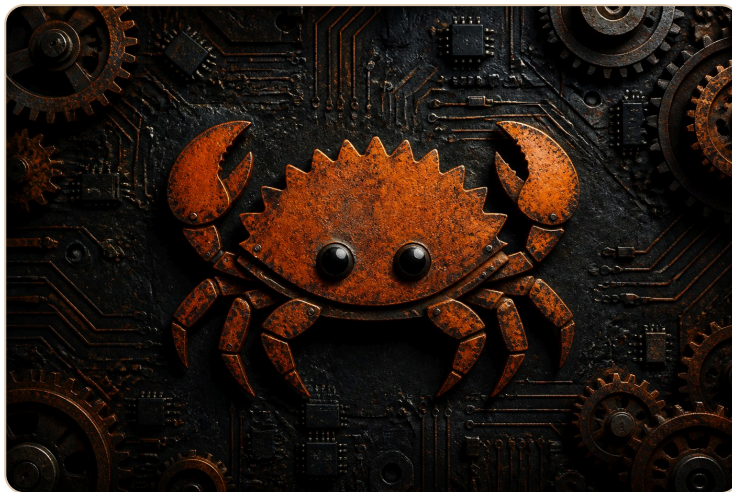
21. [Bravo !](#)

- [Tu as termine Rust - Les bases](#)
- [Ce que tu as construit](#)
- [La suite avec DanielCraft](#)
- [Un dernier conseil](#)

C'est quoi Rust ?



Rust : securite memoire + performance.



Le crabe Ferris, mascotte de Rust.

Le langage en une phrase

Rust est un langage systeme cree par Mozilla en 2010. Il garantit la securite memoire sans ramasse-miettes (garbage collector). Il est aussi rapide que C et C++ mais elimine des categories entieres de bugs.

Pourquoi apprendre Rust ?

Rust est élu "langage le plus aime" sur Stack Overflow depuis 8 ans consecutifs. Il est utilise par Microsoft, Google, Amazon, Meta et Cloudflare. Linux l'integre dans son noyau.

> **Astuce DanielCraft** - Rust est exigeant mais recompense : un code qui compile fonctionne presque toujours correctement.

Nora decouvre Rust en cherchant un langage rapide et sur. Elle ecrit un petit outil CLI qui traite des fichiers en parallele, sans crash.

A quoi ca ressemble ?

```
fn main() {  
    let nom = "Lea";  
    println!("Salut {nom}, bienvenue en Rust !");  
}
```

Petite histoire

Graydon Hoare cree Rust chez Mozilla pour construire un navigateur web plus sur. Le langage grandit, se stabilise en 2015 (version 1.0), et conquiert les systemes, le cloud et le WebAssembly.

En vrai

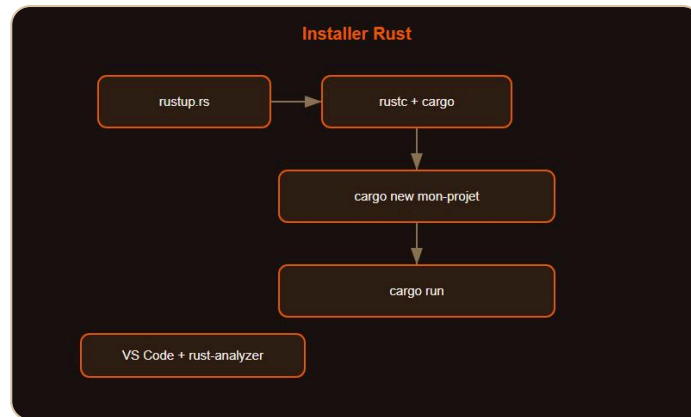
Sam utilise Rust pour creer un outil de parsing ultra-rapide. Max ecrit un serveur HTTP avec Actix. Nora compile en WebAssembly pour le navigateur.

> **Piege** - Le compilateur Rust est strict. Il refuse du code que d'autres langages accepteraient. C'est voulu : il previent les bugs avant l'execution.

A retenir

- Rust = securite memoire + performance.
- Pas de garbage collector.
- Le compilateur est ton meilleur allie.

Installer Rust



rustup + cargo : pret en 2 minutes.

Ce qu'il te faut

Rust s'installe avec `rustup`, l'outil officiel qui gere les versions du compilateur et les outils associes.

Installer rustup

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

Sur Windows : telecharge `rustup-init.exe` depuis rustup.rs.

Apres l'installation :

```
rustc --version
cargo --version
```

Cargo : le gestionnaire de projet

Cargo est l'outil central de Rust : il compile, teste, gere les dependances et publie.

```
cargo new mon-projet
cd mon-projet
cargo run
```

> **Astuce DanielCraft** - Cargo fait tout. Tu n'auras presque jamais besoin d'appeler `rustc` directement.

Installer VS Code

Installe l'extension "rust-analyzer" pour l'auto-completion, les erreurs en temps reel et la navigation dans le code.

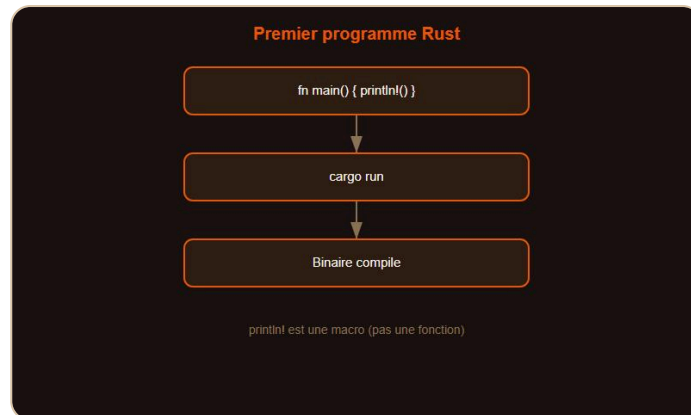
Petite histoire

Max installe Rust en 2 minutes avec `rustup`. Il tape `cargo new hello`, `cargo run`, et voit "Hello, world!" immediatement.

A retenir

- `rustup` pour installer et mettre a jour Rust.
- `cargo` pour tout : build, run, test, dependances.
- VS Code + rust-analyzer = environnement ideal.

Premier programme



`fn main()` et `println!` : premier programme.

Hello World

```
fn main() {
    println!("Bonjour le monde !");
}
```

`fn main()` est le point d'entree. `println!` est une macro (le `!` l'indique) qui affiche du texte.

Compiler et executer

```
cargo run      # Compile et execute
cargo build    # Compile seulement
cargo build --release # Compile optimise
```

> **Astuce DanielCraft** - `cargo run` pour developper. `--release` pour la production.

Affichage formate

```
let nom = "Lea";
let age = 28;
println!("Je suis {nom}, {age} ans.");
println!("{ } + { } = { }", 2, 3, 2 + 3);
```

Les commentaires

```
// Commentaire sur une ligne
/* Commentaire
   sur plusieurs lignes */
/// Documentation (genere du HTML avec cargo doc)
```

Petite histoire

Nora cree un projet avec `cargo new`, modifie `src/main.rs`, lance `cargo run`. La compilation est rapide et le message apparait.

A retenir

- `fn main()` = point d'entree.
- `println!()` pour afficher (c'est une macro).
- `cargo run` pour compiler et executer.

Les variables



let immutable, let mut mutable.

Immutabilite par default

En Rust, les variables sont immutables par default. Pour les rendre mutables, on ajoute `mut`.

```
let nom = "Max";      // Immutable
let mut score = 0;    // Mutable
score += 10;
println!("{score}");  // 10
```

Shadowing

On peut redeclarer une variable avec le meme nom :

```
let x = 5;
let x = x + 1;    // Nouveau x, pas de mut necessaire
let x = x * 2;
println!("{x}");  // 12
```

Le shadowing permet de changer le type :

```
let texte = "42";
let texte: i32 = texte.parse().unwrap();
```

Constantes

```
const TVA: f64 = 0.20;
const MAX_JOUEURS: u32 = 100;
```

> **Astuce DanielCraft** - Immutable par default, c'est le choix de Rust pour prevenir les bugs. Utilise `mut` seulement quand c'est necessaire.

Conventions

- snake_case pour les variables et fonctions : `mon_score`.
- SCREAMING_SNAKE_CASE pour les constantes : `MAX_JOUEURS`.

- PascalCase pour les types et structs : `MonType` .

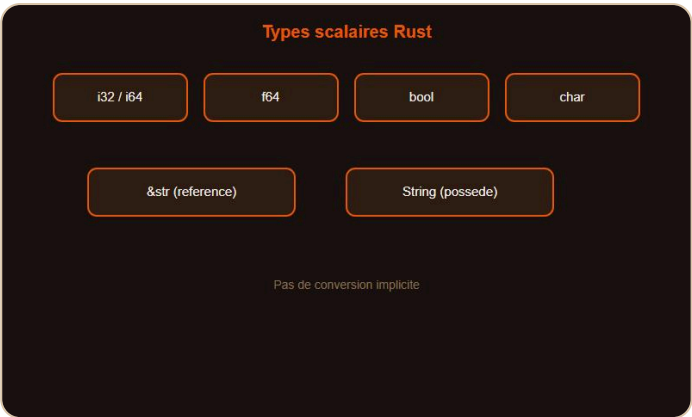
Petite histoire

Sam declare `let budget = 1800;` puis essaie `budget = 1500;` . Le compilateur refuse. Il ajoute `mut` et comprend le principe.

A retenir

- `let` = immutable, `let mut` = mutable.
- Shadowing pour redeclarer sans `mut` .
- `const` pour les constantes (type obligatoire).

Les types de donnees



i32, f64, &str, String : types scalaires.

Les types scalaires

Type	Exemples	Usage
i32	42	Entier signe 32 bits (default)
i64	9_000_000_000	Grand entier
u32	0..4 milliards	Entier non signe
f64	3.14	Flottant 64 bits (default)
bool	true / false	Booleen
char	'A', 'é'	Caractere Unicode

Les strings

```
let s1 = "Bonjour";           // &str (reference, immutable)
let s2 = String::from("Salut"); // String (sur le tas, mutable)
```

Inference de type

```
let x = 42;           // i32 par default
let y = 3.14;         // f64 par default
let z: u8 = 255;      // Type explicite
```

Conversion

```
let i = 42i32;
let f = i as f64;           // Cast
let s = i.to_string();      // Vers String
let n: i32 = "42".parse().unwrap(); // Parse
```

> **Astuce DanielCraft** - Rust ne fait aucune conversion implicite. Chaque cast est explicite et visible.

Tuples et tableaux

```
let point = (10, 20);      // Tuple
let coords = [1, 2, 3, 4]; // Tableau fixe
println!("{}", point.0);   // 10
println!("{}", coords[2]); // 3
```

Petite histoire

Nora additionne un `i32` et un `f64`. Rust refuse. Elle ajoute `as f64` et comprend : pas de magie, tout est explicite.

A retenir

- `i32` et `f64` par défaut.
- `&str` (reference) vs `String` (possède).
- Pas de conversion implicite.

Les conditions



if expression + match exhaustif.

if / else

```
let age = 17;
if age >= 18 {
    println!("Majeur");
} else {
    println!("Mineur");
}
```

Pas de parenthèses, accolades obligatoires.

if comme expression

```
let statut = if age >= 18 { "Majeur" } else { "Mineur" };
println!("{statut}");
```

En Rust, `if` est une expression qui retourne une valeur.

else if

```
let note = 14;
let mention = if note >= 16 {
    "Tres bien"
} else if note >= 12 {
    "Bien"
} else if note >= 10 {
    "Passable"
} else {
    "Insuffisant"
};
```


match

```
let jour = "lundi";
match jour {
  "lundi" | "mardi" => println!("Debut de semaine"),
  "vendredi" => println!("Presque le weekend"),
  _ => println!("Autre jour"),
}
```

`match` est exhaustif : il faut couvrir tous les cas (ou utiliser `_`).

> **Astuce DanielCraft** - `match` est plus puissant que `switch`. Il gere les patterns, les ranges, les destructurations.

Petite histoire

Max ecrit un match sur une enum `Direction`. Le compilateur lui dit qu'il a oublie `Nord`. Il ajoute le cas et le code compile.

A retenir

- `if` est une expression (retourne une valeur).
- Pas de parentheses, accolades obligatoires.
- `match` est exhaustif et tres puissant.

Les boucles



loop, while, for in : les boucles Rust.

loop : boucle infinie

```
let mut compteur = 0;
loop {
    compteur += 1;
    if compteur == 5 { break; }
}
```

`loop` peut retourner une valeur :

```
let resultat = loop {
    compteur += 1;
    if compteur == 10 { break compteur * 2; }
};
```

while

```
let mut n = 0;
while n < 5 {
    println!("{n}");
    n += 1;
}
```

for et ranges

```
for i in 0..5 {
    println!("{i}"); // 0, 1, 2, 3, 4
}

for i in 0..=5 {
    println!("{i}"); // 0, 1, 2, 3, 4, 5 (inclusif)
}
```

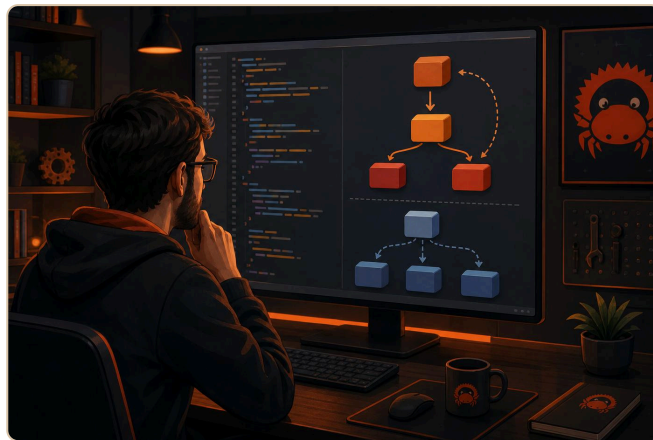
Parcourir une collection

```
let fruits = vec!["pomme", "banane", "cerise"];
for fruit in &fruits {
    println!("{fruit}");
}
```

> **Astuce DanielCraft** - `&fruits` emprunte la collection. Sans `&`, la boucle consommerait le vecteur.

break et continue

```
for i in 0..10 {
    if i == 5 { break; }
    if i % 2 == 0 { continue; }
    println!("{i}"); // 1, 3
}
```



Sam decouvre les iterateurs Rust.

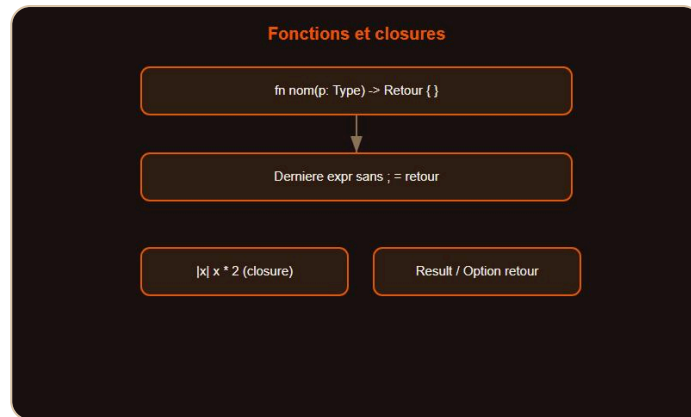
Petite histoire

Sam utilise `for i in 0..100` pour calculer une somme. Pas de variable d'index separee, pas d'oubli d'incrementation.

A retenir

- `loop` = infini, peut retourner une valeur.
- `for i in 0..n` pour les ranges.
- `&collection` pour emprunter dans un `for`.

Les fonctions



Fonctions et closures.

Declarer une fonction

```
fn saluer(prenom: &str) {
    println!("Bonjour {prenom} !");
}

fn main() {
    saluer("Nora");
}
```

Retourner une valeur

```
fn additionner(a: i32, b: i32) -> i32 {
    a + b // Pas de ; = expression retournee
}
```

En Rust, la dernière expression (sans `;`) est la valeur de retour.

Return explicite

```
fn diviser(a: f64, b: f64) -> f64 {
    if b == 0.0 {
        return 0.0; // Retour anticipe
    }
    a / b
}
```

Closures

```
let doubler = |x: i32| x * 2;
println!("{}", doubler(5)); // 10

let mut nombres = vec![3, 1, 2];
nombres.sort_by(|a, b| a.cmp(b));
```

> **Astuce DanielCraft** - Pas de `;` à la fin = expression retournee. C'est une convention forte en Rust.

Petite histoire

Max écrit une fonction `calculer_ttc(prix: f64) -> f64`. Il oublie le `;` à la fin, et ça marche. Il comprend le principe des expressions.

A retenir

- `fn nom(params) -> TypeRetour { ... }`
- Dernière expression sans `;` = retour.
- Closures avec `|params| expression`.

L'ownership



Ownership : le coeur de Rust.

Le concept central de Rust

L'ownership (possession) est ce qui rend Rust unique. Chaque valeur a un seul propriétaire. Quand le propriétaire sort du scope, la valeur est libérée.

```

{
    let s = String::from("Bonjour");
    // s est valide ici
} // s est libere automatiquement
  
```

Le déplacement (move)

```

let s1 = String::from("Salut");
let s2 = s1; // s1 est "deplace" vers s2
// println!("{s1}"); // Erreur ! s1 n'existe plus
println!("{s2}");    // OK
  
```

Le clonage

```

let s1 = String::from("Salut");
let s2 = s1.clone(); // Copie profonde
println!("{s1} {s2}"); // OK
  
```

Les references (emprunt)

```

fn longueur(s: &String) -> usize {
    s.len()
}

let mot = String::from("Bonjour");
let len = longueur(&mot); // Emprunte sans prendre possession
println!("{mot} fait {len} caracteres");
  
```

References mutables

```
fn ajouter_monde(s: &mut String) {  
    s.push_str(" monde !");  
}  
  
let mut salut = String::from("Bonjour");  
ajouter_monde(&mut salut);
```

> **Piege** - Une seule reference mutable OU plusieurs references immutables a la fois. Jamais les deux. C'est ce qui previent les data races.



Securite memoire sans garbage collector.

Petite histoire

Nora passe un `String` a une fonction. Le compilateur refuse de l'utiliser ensuite. Elle ajoute `&` et comprend l'emprunt.

A retenir

- Chaque valeur a un proprietaire unique.
- Le deplacement transfere la possession.
- `&` emprunte, `&mut` emprunte de facon mutable.
- Une seule `&mut` OU plusieurs `&` a la fois.

Les structs



Structs et impl.

Définir une struct

```

struct Animal {
    nom: String,
    age: u32,
}

fn main() {
    let chat = Animal {
        nom: String::from("Felix"),
        age: 3,
    };
    println!("{}", chat.nom, chat.age);
}
  
```

Methodes avec impl

```

impl Animal {
    fn se_presenter(&self) -> String {
        format!("Je suis {}, {} ans.", self.nom, self.age)
    }

    fn new(nom: &str, age: u32) -> Self {
        Self {
            nom: String::from(nom),
            age,
        }
    }
}

let chat = Animal::new("Felix", 3);
println!("{}", chat.se_presenter());
  
```

`&self` emprunte la struct. `Self` est un alias pour le type.

Tuple structs

```
struct Point(f64, f64);
let p = Point(1.0, 2.5);
println!("{}", p.0, p.1);
```

Derive

```
#[derive(Debug, Clone)]
struct Produit {
    nom: String,
    prix: f64,
}

let p = Produit { nom: String::from("Clavier"), prix: 49.99 };
println!("{:?}", p); // Debug
```

> **Astuce DanielCraft** - `#[derive(Debug)]` permet d'afficher une struct avec `{:?}`. Indispensable pour le debug.

Petite histoire

Max modele un `Produit` avec `impl`. Il ajoute `#[derive(Debug)]` et peut inspecter ses objets facilement.

A retenir

- `struct` pour les donnees, `impl` pour les methodes.
- `&self` pour emprunter, `Self` pour le type.
- `#[derive(Debug)]` pour l'affichage.

Enums et pattern matching



Enums + Option + match.

Les enums

Une enum définit un type avec plusieurs variantes.

```
enum Direction {
    Nord,
    Sud,
    Est,
    Ouest,
}

let dir = Direction::Nord;
```

Enums avec données

```
enum Message {
    Quitter,
    Texte(String),
    Position { x: i32, y: i32 },
}

let msg = Message::Texte(String::from("Bonjour"));
```

match avec enums

```
match dir {
    Direction::Nord => println!("Vers le nord"),
    Direction::Sud => println!("Vers le sud"),
    Direction::Est => println!("Vers l'est"),
    Direction::Ouest => println!("Vers l'ouest"),
}
```

Option<T>

Rust n'a pas de `null`. À la place, `Option<T>` :

```
fn trouver(liste: &[i32], cible: i32) -> Option<usize> {
    liste.iter().position(|&x| x == cible)
}

match trouver(&[1, 2, 3], 2) {
    Some(index) => println!("Trouve a l'index {index}"),
    None => println!("Non trouve"),
}
```

if let

```
if let Some(i) = trouver(&[1, 2, 3], 2) {
    println!("Index : {i}");
}
```

> **Astuce DanielCraft** - `Option` et `Result` remplacent `null` et les exceptions. Le compilateur t'oblige a gerer les deux cas.

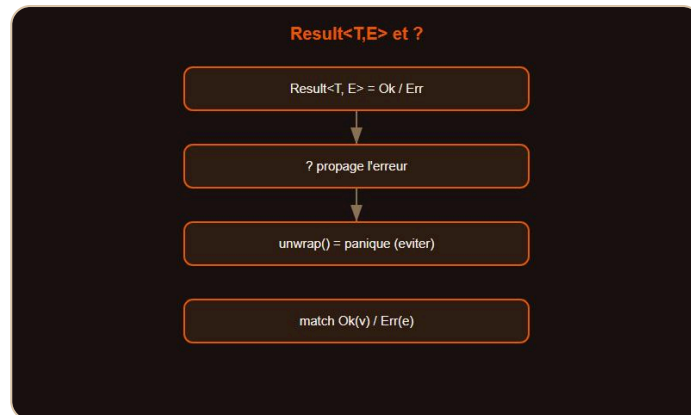
Petite histoire

Nora cherche un element dans un vecteur. La fonction retourne `Option`. Le compilateur l'oblige a gerer le cas "absent". Plus de crash surprise.

A retenir

- `enum` pour les variantes.
- `match` est exhaustif.
- `Option<T>` = `Some(valeur)` ou `None`.
- Pas de `null` en Rust.

Gerer les erreurs



Result<T,E> et l'opérateur ?.

Result<T, E>

Rust utilise `Result` pour les opérations qui peuvent échouer :

```

use std::fs;

fn lire_fichier(chemin: &str) -> Result<String, std::io::Error> {
    fs::read_to_string(chemin)
}

match lire_fichier("data.txt") {
    Ok(contenu) => println!("{contenu}"),
    Err(e) => println!("Erreur : {e}"),
}
  
```

L'opérateur ?

```

fn lire_et_compter(chemin: &str) -> Result<usize, std::io::Error> {
    let contenu = fs::read_to_string(chemin)?; // Propage l'erreur
    Ok(contenu.len())
}
  
```

`?` retourne l'erreur automatiquement si le Result est Err.

unwrap et expect

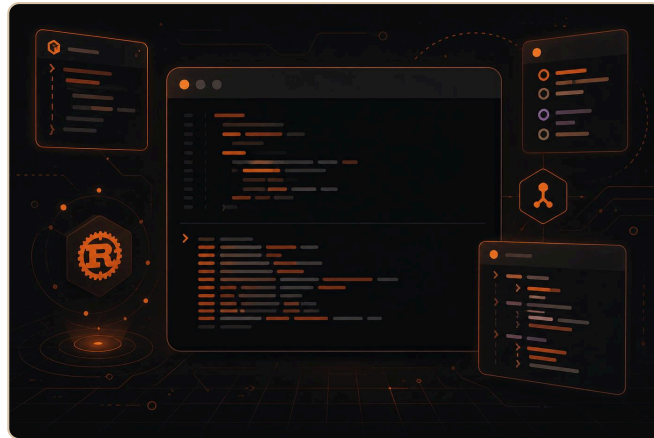
```

let n: i32 = "42".parse().unwrap(); // Panique si erreur
let n: i32 = "42".parse().expect("Pas un nombre"); // Message custom
  
```

> **Piege** - `unwrap()` panique (crash). Utilise-le seulement dans les tests ou quand tu es certain du résultat.

Créer ses propres erreurs

```
fn retirer(solde: f64, montant: f64) -> Result<f64, String> {  
    if montant > solde {  
        return Err(String::from("Solde insuffisant"));  
    }  
    Ok(solde - montant)  
}
```



Max refactorise avec Result.

Petite histoire

Max utilise `unwrap()` partout. Son programme panique sur un fichier manquant. Il remplace par `?` et gère proprement l'erreur.

A retenir

- `Result<T, E>` = `Ok(valeur)` ou `Err(erreur)`.
- `?` propage l'erreur élégamment.
- `unwrap()` = panique. À éviter en production.

Mini-projet : outil CLI de notes



Mini-projet : outil CLI de notes.

L'objectif

On crée un outil en ligne de commande qui permet d'ajouter, lister et chercher des notes. Sauvegarde en fichier texte simple.

Structure (src/main.rs)

```

use std::fs;
use std::io::{self, Write};

const FICHIER: &str = "notes.txt";

fn charger() -> Vec<String> {
    fs::read_to_string(FICHIER)
        .unwrap_or_default()
        .lines()
        .filter(|l| !l.is_empty())
        .map(String::from)
        .collect()
}

fn sauvegarder(notes: &[String]) {
    let contenu = notes.join("\n");
    fs::write(FICHIER, contenu).expect("Impossible d'ecrire");
}

fn lire_ligne(prompt: &str) -> String {
    print!("{prompt}");
    io::stdout().flush().unwrap();
    let mut buf = String::new();
    io::stdin().read_line(&mut buf).unwrap();
    buf.trim().to_string()
}

fn main() {
    let mut notes = charger();
    loop {
        println!("\n1. Ajouter 2. Lister 3. Chercher 4. Quitter");
        match lire_ligne("> ").as_str() {
            "1" => {
                let note = lire_ligne("Note : ");
                if !note.is_empty() {

```

```

        notes.push(note);
        sauvegarder(&notes);
        println!("Ajoutée.");
    }
}
"2" => {
    if notes.is_empty() {
        println!("Aucune note.");
    } else {
        for (i, n) in notes.iter().enumerate() {
            println!("  {}. {n}", i + 1);
        }
    }
}
"3" => {
    let terme = lire_ligne("Recherche : ").to_lowercase();
    for n in &notes {
        if n.to_lowercase().contains(&terme) {
            println!("  {n}");
        }
    }
}
"4" => { println!("A bientôt !"); break; }
_ => {}
}
}
}

```

Ce que tu apprends

- `std::fs` pour lire/écrire des fichiers.
- `Vec<String>` et les itérateurs.
- `match` sur des chaînes.
- Ownership : `¬es` pour emprunter.

> **Astuce DanielCraft** - Commence par le menu, puis ajoute une commande à la fois.

A retenir

- Ownership et emprunt en action.
- Itérateurs pour transformer les données.
- `match` pour le menu.

Ce qu'il faut retenir



Carte des notions Rust.

Les briques essentielles

Concept	Resume
Variables	<code>let</code> immutable, <code>let mut</code> mutable
Types	<code>i32</code> , <code>f64</code> , <code>bool</code> , <code>&str</code> , <code>String</code>
Conditions	<code>if</code> expression, <code>match</code> exhaustif
Boucles	<code>loop</code> , <code>while</code> , <code>for in</code>
Fonctions	Derniere expr = retour, closures
Ownership	Un propriétaire, move, & emprunt
Structs	Donnees + impl methodes
Enums	Variantes, Option, pattern matching
Erreurs	<code>Result<T,E></code> , <code>?</code> , pas d'exceptions

La methode

1. Lis le chapitre.
 2. Tape les exemples.
 3. Fais compiler (le compilateur enseigne).
 4. Fais les ateliers.
 5. Reviens quand tu bloques.
- > **Astuce DanielCraft** - Le compilateur Rust est le meilleur professeur. Lis ses messages attentivement.

Ce qui vient apres

- Lifetimes et references avancees.

- Traits et generiques.
- Concurrency avec threads et async.
- Crates et Cargo avance.
- WebAssembly.

A retenir

- Rust est exigeant mais recompense.
- Un code qui compile est un code solide.

Atelier : variables et types



Atelier : variables et types.

Exercice 1 : carte d'identite

```
let prenom = "Sam";
let age = 22;
let ville = "Nantes";
println!("Je suis {prenom}, {age} ans, je vis a {ville}.");
```

Exercice 2 : shadowing

```
let x = 5;
let x = x + 1;
let x = x * 2;
println!("{x}"); // 12
```

Exercice 3 : conversion

```
let texte = "42";
let nombre: i32 = texte.parse().unwrap();
let flottant = nombre as f64;
println!("{flottant}"); // 42.0
```

Defi : temperature

```
let celsius = 37.0_f64;
let fahrenheit = celsius * 9.0 / 5.0 + 32.0;
println!("{celsius}°C = {fahrenheit:.1}°F");
```

> **Astuce DanielCraft** - `:.1` formate avec 1 decimale. `:.2` avec 2.

A retenir

- `let` pour immutable, `let mut` pour mutable.
- Shadowing = redeclarer sans `mut`.
- `as` pour les casts entre types numeriques.

Atelier : fonctions



Atelier : fonctions.

Exercice 1 : salutation

```
fn saluer(nom: &str, heure: u32) -> String {
    if heure < 12 {
        format!("Bonjour {nom} !")
    } else if heure < 18 {
        format!("Bon apres-midi {nom} !")
    } else {
        format!("Bonsoir {nom} !")
    }
}
```

Exercice 2 : moyenne

```
fn moyenne(notes: &[f64]) -> f64 {
    if notes.is_empty() { return 0.0; }
    notes.iter().sum::<f64>() / notes.len() as f64
}
```

Exercice 3 : mot de passe valide

```
fn mdp_valide(mdp: &str) -> bool {
    mdp.len() >= 8 && mdp.chars().any(|c| c.is_ascii_digit())
}
```

Exercice 4 : division securisee

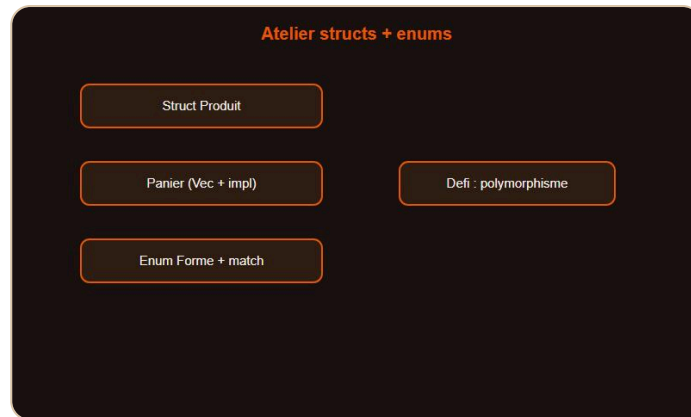
```
fn diviser(a: f64, b: f64) -> Result<f64, String> {
    if b == 0.0 {
        Err(String::from("Division par zero"))
    } else {
        Ok(a / b)
    }
}
```

> **Astuce DanielCraft** - `&[f64]` (slice) est plus flexible que `&Vec<f64>`.

A retenir

- `&str` et `&[T]` pour les emprunts.
- Iterateurs (`.iter()`, `.sum()`, `.any()`) au lieu de boucles.
- `Result` pour les erreurs.

Atelier : structs et enums



Atelier : structs et enums.

Exercice 1 : struct Produit

```
#[derive(Debug)]
struct Produit {
    nom: String,
    prix: f64,
}

impl Produit {
    fn new(nom: &str, prix: f64) -> Self {
        Self { nom: String::from(nom), prix }
    }
    fn afficher(&self) -> String {
        format!("{}", - {:.2} EUR", self.nom, self.prix)
    }
}
```

Exercice 2 : panier

```
struct Panier {
    produits: Vec<Produit>,
}

impl Panier {
    fn new() -> Self { Self { produits: vec![] } }
    fn ajouter(&mut self, p: Produit) { self.produits.push(p); }
    fn total(&self) -> f64 {
        self.produits.iter().map(|p| p.prix).sum()
    }
}
```

Exercice 3 : enum Forme

```
use std::f64::consts::PI;

enum Forme {
    Cercle(f64),
    Rectangle(f64, f64),
}

impl Forme {
    fn aire(&self) -> f64 {
        match self {
            Forme::Cercle(r) => PI * r * r,
            Forme::Rectangle(l, h) => l * h,
        }
    }
}
```

> **Astuce DanielCraft** - `&mut self` pour modifier, `&self` pour lire.

A retenir

- `#[derive(Debug)]` pour inspecter.
- `impl` separe donnees et comportement.
- `match` sur les enums pour le polymorphisme.

Erreurs classiques en Rust



Pieges classiques Rust.

1. Utiliser une valeur apres un move

```
let s = String::from("Bonjour");
let s2 = s;
// println!("{s}"); // Erreur : s a ete deplace
```

Solution : cloner ou emprunter avec `&`.

2. Reference mutable + immutable

```
let mut v = vec![1, 2, 3];
let r = &v[0];
v.push(4); // Erreur : v est emprunte immuablement par r
println!("{r}");
```

3. Oublier le point-virgule

```
fn double(x: i32) -> i32 {
    x * 2; // Le ; transforme l'expression en statement (retourne ())
}
```

Enleve le `;` pour retourner la valeur.

4. unwrap() en production

```
let fichier = fs::read_to_string("absent.txt").unwrap(); // Panique !
```

Utilise `?` ou `match` a la place.

5. Confusion &str vs String

```
fn saluer(nom: String) {} // Prend possession
fn saluer(nom: &str) {} // Emprunte seulement (prefere)
```

6. Oublier mut

```
let v = vec![1, 2, 3];  
v.push(4); // Erreur : v n'est pas mutable
```

> **Astuce DanielCraft** - Le compilateur Rust donne des messages d'erreur excellents. Lis-les en entier, ils contiennent souvent la solution.

A retenir

- Move != copie. Emprunte avec `&` quand possible.
- Pas de `;` pour retourner une valeur.
- `?` au lieu de `unwrap()` en production.

Bonnes pratiques



cargo fmt, clippy, test.

Conventions Rust

- **snake_case** : variables, fonctions, modules.
- **PascalCase** : types, structs, enums, traits.
- **SCREAMING_SNAKE_CASE** : constantes.
- **Clippy** : linter officiel, `cargo clippy`.

Structure de projet

```
mon-projet/
  Cargo.toml      # Dependances et metadata
  src/
    main.rs       # Point d'entree
    lib.rs        # Bibliotheque (optionnel)
```

Gestion des erreurs

```
// Prefere ceci :
let contenu = fs::read_to_string("data.txt");

// Plutot que :
let contenu = fs::read_to_string("data.txt").unwrap();
```

Documentation

```
/// Calcule le prix TTC a partir du HT.  
///  
/// # Exemples  
/// ```  
/// let ttc = calculer_ttc(100.0);  
/// assert_eq!(ttc, 120.0);  
/// ```  
fn calculer_ttc(ht: f64) -> f64 {  
    ht * 1.20  
}
```

Les doc-tests (`///`) sont executes par `cargo test` .

Outils

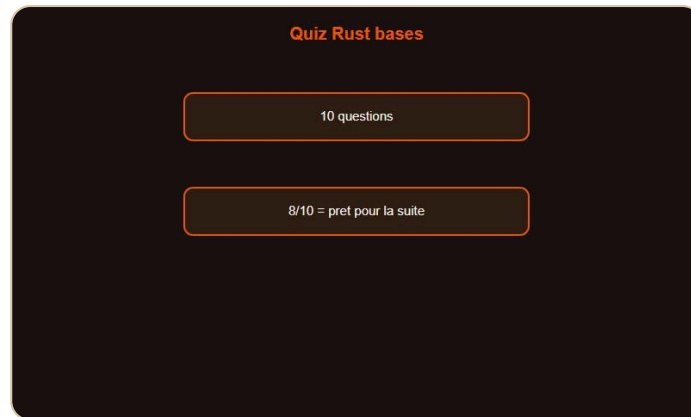
- `cargo fmt` : formatage automatique.
- `cargo clippy` : suggestions d'amelioration.
- `cargo test` : tests unitaires et doc-tests.

> **Astuce DanielCraft** - Lance `cargo clippy` avant chaque commit. Il detecte les patterns non idiomatiques.

A retenir

- `cargo fmt` + `cargo clippy` + `cargo test` = qualite.
- `?` pour propager les erreurs.
- Doc-tests dans les commentaires `///` .

Quiz Rust - Les bases



Quiz Rust bases.

Teste tes connaissances

Q1. Les variables en Rust sont par default :

A) Mutables B) Immutables C) Globales D) Dynamiques

Q2. Quel operateur propage une erreur Result ?

A) ! B) ?? C) ? D) throw

Q3. `String::from("texte")` cree :

A) Une reference &str B) Un String sur le tas C) Un tableau de char D) Une constante

Q4. Que se passe-t-il apres un move ?

A) La variable est copiee B) La variable originale est invalide C) Les deux variables partagent la memoire
D) Rien

Q5. `match` en Rust est :

A) Optionnel B) Exhaustif (tous les cas requis) C) Identique a switch D) Reserve aux nombres

Q6. `Option<T>` remplace :

A) Les exceptions B) null C) Les generiques D) Les boucles

Q7. Combien de references mutables simultanees sont autorisees ?

A) Illimite B) 2 C) 1 D) 0

Q8. `cargo clippy` sert a :

A) Compiler B) Tester C) Detecter les patterns non idiomatiques D) Deployer

Q9. La derniere expression sans `;` dans une fonction :

A) Provoque une erreur B) Est ignoree C) Est la valeur de retour D) Est un commentaire

Q10. `&str` est :

A) Un String mutable B) Une reference immutable vers du texte C) Un tableau de bytes D) Un type numerique

Reponses

1-B, 2-C, 3-B, 4-B, 5-B, 6-B, 7-C, 8-C, 9-C, 10-B

> **Astuce DanielCraft** - 8/10 ou plus ? Tu es pret pour Rust intermediaire.

Bravo !

Bravo. Tu codes en Rust.

Tu as termine Rust - Les bases

Félicitations. Tu sais écrire des programmes Rust sûrs, performants et idiomatiques. Tu maîtrises les variables, les types, les conditions, les boucles, les fonctions, l'ownership, les structs, les enums, Option, Result et la gestion d'erreurs.

Ce que tu as construit

- Un premier programme compile.
- Des fonctions avec gestion d'erreurs.
- Un outil CLI de notes complet.
- Des exercices sur chaque notion.

La suite avec DanielCraft

Le livre **Rust - Intermediaire** couvrira :

- Lifetimes et références avancées.
- Traits et génériques.
- Smart pointers (Box, Rc, Arc).
- Concurrency avec threads et async.
- Crates populaires (serde, tokio, reqwest).

> **Astuce DanielCraft** - Rust est un investissement. Plus tu l'utilises, plus tu apprécies sa rigueur. Choisis un projet CLI et pousse-le.

Un dernier conseil

Rust te rend meilleur dans tous les langages. Comprendre l'ownership, les lifetimes et les emprunts change ta façon de penser le code. Chaque programme qui compile est une victoire.

Tu as les bases. Maintenant, compile.

Tu as les briques. Maintenant, compile.